

✓ 1. Generate **prime numbers** in the range 0-20 using list comprehension

```
def is_prime (n):
```

```
    if n < 1:
```

```
        return False
```

```
    if n <= 3:
```

```
        return True
```

```
    if n % 2 == 0 or n % 3 == 0:
```

```
        return False
```

```
    i = 5
```

```
    while i * i <= n:
```

```
        if n % i == 0 or n % (i + 2) == 0:
```

```
            return False
```

```
        i += 6
```

```
    return True
```

```
prime = [num for num in range (1,21) if is_prime(num)]
```

```
print("Prime numbers between 0 and 20 are:", prime)
```

2. Write a recursive program to find the **Greatest Common Divisor** of the two integer numbers.

```
def gcd(a, b):
```

```
    if b == 0:
```

```
        return a
```

```
    else:
```

```
        return gcd(b, a % b)
```

```
num1 = int(input("Enter first number: "))
```

```
num2 = int(input("Enter second number: "))
```

```
result = gcd(num1, num2)
```

```
print(f"The greatest common divisor of {num1} and {num2}  
is: {result}")
```

3. Write a program to read, add, display and subtract two matrices using OOP

```
class Matrix:
```

```
    def __init__(self, rows, cols):
```

```
        self.rows = rows
```

```
        self.cols = cols
```

```
        self.data = [[0.0 for _ in range(cols)] for _ in range(rows)]
```

```
    def input_matrix(self, name):
```

```
        print(f"Enter elements for matrix {name}:")
```

```
        for i in range(self.rows):
```

```
            for j in range(self.cols):
```

```
                self.data[i][j] = float(input(f"Element[{i}][{j}]:"))
```

```
    def __add__(self, other): # Corrected to __add__
```

```
        if self.rows != other.rows or self.cols != other.cols:
```

```
            raise ValueError("Matrices must have the same  
dimensions for addition.")
```

```
        result = Matrix(self.rows, self.cols)
```

```
        for i in range(self.rows):
```

```
            for j in range(self.cols):
```

```

        result.data[i][j] = self.data[i][j] + other.data[i][j]
    return result # Moved return outside the loop

def __sub__(self, other): # Corrected to __sub__
    if self.rows != other.rows or self.cols != other.cols:
        raise ValueError("Matrices must have the same
dimensions for subtraction.")
    result = Matrix(self.rows, self.cols)
    for i in range(self.rows):
        for j in range(self.cols):
            result.data[i][j] = self.data[i][j] - other.data[i][j]
    return result # Moved return outside the loop

def display(self, name):
    print(f"{name}:")
    for row in self.data:
        print(" ".join(map(str, row)))
    print()

def main():
    rows = int(input("Enter the number of rows: "))

```

```
cols = int(input("Enter the number of columns: "))
matrix1 = Matrix(rows, cols)
matrix2 = Matrix(rows, cols)
matrix1.input_matrix("matrix1")
matrix2.input_matrix("matrix2")
matrix3 = matrix1 + matrix2 # Using __add__
matrix4 = matrix1 - matrix2 # Using __sub__
matrix3.display(" Addition of matrix1 and matrix2 =
matrix3 ")
matrix4.display(" subtraction of matrix1-matrix2 =
matrix4")

if __name__ == "__main__":
    main()
```

4. Write a program to calculate area, perimeter and height of the triangle using the concept of classes and objects

```
import math
```

```
class Triangle:
```

```
    def __init__(self, side1, side2, side3):
```

```
        self.side1 = side1
```

```
        self.side2 = side2
```

```
        self.side3 = side3
```

```
    def calculate_perimeter(self):
```

```
        return self.side1 + self.side2 + self.side3
```

```
    def calculate_area(self):
```

```
        # Calculate the semi-perimeter
```

```
        s = self.calculate_perimeter() / 2
```

```
        # Calculate the area using Heron's formula
```

```
        area = math.sqrt(s * (s - self.side1) * (s - self.side2) * (s - self.side3))
```

```
        return area
```

```
def calculate_height(self):  
    # Assuming side1 is the base  
    area = self.calculate_area()  
    height = (2 * area) / self.side1  
    return height
```

```
# Get triangle side lengths from the user  
side1 = float(input("Enter the length of side 1: "))  
side2 = float(input("Enter the length of side 2: "))  
side3 = float(input("Enter the length of side 3: "))
```

```
# Create a Triangle object  
triangle = Triangle(side1, side2, side3)
```

```
# Calculate and print the perimeter, area, and height  
perimeter = triangle.calculate_perimeter()  
area = triangle.calculate_area()  
height = triangle.calculate_height()
```

```
print("Perimeter:", perimeter)  
print("Area:", area)
```

```
print("Height (with side 1 as base):", height)
```


5. Create a module that holds the functions for finding the number of different ways the various combinations of words can be given with a single word.

```
from collections import Counter
```

```
import math
```

```
def word_combinations(word):
```

```
    if not word: #handle empty or none input
```

```
        return 0
```

```
    counts =Counter(word)
```

```
    total_letters =len(word)
```

```
    if len(counts)==1: #handle cases where there is only on  
unique character
```

```
        return 1
```

```
    numerator =math.factorial(total_letters)
```

```
    denominator =1
```

```
    for count in counts.values():
```

```
        denominator *= math.factorial(count)
```

```
return numerator //denominator
```

```
def main():
```

```
    """Demonstrates the word_combinations function with  
    various examples."""
```

```
    test_words  
    =["apple","banana","a","", "statistics","mississippi","noon","aa  
    bbcc"]
```

```
    for word in test_words:
```

```
        num_combinations =word_combinations(word)
```

```
        print(f"Word:{word}', Number of combinations:  
{num_combinations}")
```

```
if __name__ == "__main__":
```

```
    main()
```

6. Construct a python script that finds the lam for the input numbers through the command line arguments.

```
import sys
```

```
n = len(sys.argv)
```

```
print("Total arguments passed:", n)
```

```
print("\nName of Python script:", sys.argv[0])
```

```
print("Arguments Passed:", end=" ")
```

```
for i in range(1, n):
```

```
    print(sys.argv[i], end=" ")
```

```
sum_ = 0
```

```
for i in range(1, n):
```

```
    sum_ += int(sys.argv[i])
```

```
print("\nSum Result:", sum_)
```

7. Write a python program to demonstrate raising exceptions for a ZeroDivisionError

```
def divide_numbers(a, b):  
    if b == 0:  
        raise ZeroDivisionError("Cannot divide by zero")  
    return a / b  
  
try:  
    num1 = float(input("Enter the numerator: "))  
    num2 = float(input("Enter the denominator: "))  
  
    result = divide_numbers(num1, num2)  
    print("Result of division:", result)  
  
except ZeroDivisionError as e:  
    print("Error:", e)
```

8. Write a python script for handling FileNotFoundError Exceptions

try:

 with open("non_existent_file.txt", "r") as file: # 'r' mode to
 read an existing file

 content = file.read()

 print("File content:", content)

except FileNotFoundError:

 print("Error: The file does not exist.")

9. Write a python program to implement Pandas Series with labels.

1 import pandas as pd

2 # Ask user how many items to input

n = int(input("How many fruit entries do you want to add? "))

3 # Initialize an empty dictionary

data = {}

4 # Collect user inputs and populate the dictionary

for i in range(n):

fruit_name = input(f"Enter the name of fruit {i+1}: ")

fruit_quantity = int(input(f"Enter the quantity of
{fruit_name}: "))

data[fruit_name] = fruit_quantity

5 fruit_series = pd.Series(data)

print("\nPandas Series with labels:")

print(fruit_series)

✓ 10. Using Numpy, find the five-point summary of the following list of numbers. A=[10 40 60 100 120 140]

```
import numpy as np # Import NumPy library
```

```
# Define the given list of numbers as a NumPy array
```

```
A = np.array([10, 40, 60, 100, 120, 140])
```

```
# Calculate the five-number summary
```

```
min_value = np.min(A) # Minimum
```

```
q1 = np.percentile(A, 25) # First Quartile (Q1)
```

```
median = np.median(A) # Median (Q2)
```

```
q3 = np.percentile(A, 75) # Third Quartile (Q3)
```

```
max_value = np.max(A) # Maximum
```

```
# Print the five-number summary
```

```
print("Five-Number Summary:")
```

```
print("Minimum:", min_value)
```

```
print("First Quartile (Q1):", q1)
```

```
print("Median (Q2):", median)
```

```
print("Third Quartile (Q3):", q3)
```

```
print("Maximum:", max_value)
```